

Ai-VendorHub Auth Service Documentation

Backend + Frontend Coverage Report | Auth Service Only

This PDF documents the Auth service backend APIs, the React frontend pages created for those APIs, and the remaining frontend-only gaps. It focuses only on Auth service, not product/cart/order/payment services.

1. Service Overview

Item	Details
Backend Auth Service	Express.js service mounted under /api , normally running at http://localhost:3001/api .
Frontend Auth UI	React + Vite frontend normally running at http://localhost:5173 .
Auth Mechanism	JWT access token + refresh token cookie. Frontend also stores access token in localStorage for Authorization header.
Session Handling	Refresh-token sessions stored in Redis with max 5 active sessions per user.
Security Middleware	Helmet, CORS, JSON parser, cookie parser, Passport initialization, login/register/email rate limits.
Events	Publishes user.created and user.updated to RabbitMQ queues.
Email Features	Email verification and password reset via email service; dev token returned outside production for local testing.

2. Frontend Pages Created

Frontend Route	Purpose	Status
/login	Buyer/Seller login, role toggle, Google login entry.	Done
/register	Buyer/Seller signup, role-based form, Google signup entry.	Done
/forgot-password	Request password reset email and show local dev reset link if backend returns token.	Done
/reset-password/:token	Reset password using token from email/dev link.	Done
/verify-email/:token	Verify email token and update account session.	Done
/auth/success	Google OAuth success bridge page; refreshes user then redirects to profile.	Done
/profile	Account dashboard: profile update, addresses, security, sessions, payments info, saved/support UI.	Done

3. Backend API To Frontend Mapping

Backend API	Method	Feature	Frontend?	Frontend Usage
/api/auth/register	POST	Buyer registration	Yes	/register
/api/auth/register/seller	POST	Seller registration	Yes	/register

Backend API	Method	Feature	Frontend?	Frontend Usage
/api/auth/login	POST	Buyer login	Yes	/login
/api/auth/login/seller	POST	Seller login	Yes	/login
/api/auth/google	GET	Start Google OAuth	Yes	Google button in login/register
/api/auth/google/callback	GET	Google OAuth callback	Indirect	Google calls backend; redirects to /auth/success
/api/auth/me	GET	Current user/seller	Yes	AuthContext + protected routes + profile
/api/auth/refresh	POST	Refresh access token	Yes	Axios 401 interceptor + profile sync button
/api/auth/logout	POST	Logout current session	Yes	Profile logout button
/api/auth/logout-all	POST	Logout all devices	Yes	Profile security section
/api/auth/users/me	PATCH	Update profile	Yes	Profile edit form
/api/auth/verify-email/request	POST	Request email verification	Yes	Profile security/email prompt
/api/auth/verify-email/:token	POST	Verify email	Yes	/verify-email/:token
/api/auth/password/forgot	POST	Forgot password email	Yes	/forgot-password
/api/auth/password/reset/:token	POST	Reset password	Yes	/reset-password/:token
/api/auth/users/me/addresses	POST	Add address	Yes	Profile address modal
/api/auth/users/me/addresses/:addressId	DELETE	Delete address	Yes	Profile address cards

4. APIs Existing But No Separate Frontend Needed

API	Reason	Frontend Status
GET /api	Health-check endpoint only. Useful for developer/server status, not required for user auth flow.	Optional
GET /api/auth/logout	Alias for logout. Frontend already uses POST /api/auth/logout, which is better for state-changing action.	Not needed
GET /api/auth/verify-email/:token	Alias for verify email. Frontend uses POST /api/auth/verify-email/:token.	Not needed
GET /api/auth/users/me/addresses	Address list endpoint exists, but frontend currently receives addresses from GET /api/auth/me and updates state after add/delete.	Optional

5. Request Bodies For Main Auth APIs

API	Example Body
POST /api/auth/register	{ username, email, password, fullName: { firstName, lastName }, role: 'user', address: { addressLine, city, state, pincode, phone } }
POST /api/auth/register/seller	{ username, email, password, fullName: { firstName, lastName } }
POST /api/auth/login	{ email, password } OR { username, password }
POST /api/auth/login/seller	{ email, password } OR { username, password }
PATCH /api/auth/users/me	{ username, email, fullName: { firstName, lastName } }
POST /api/auth/password/forgot	{ email }
POST /api/auth/password/reset/:token	{ password }
POST /api/auth/users/me/addresses	{ addressLine, city, state, pincode, phone }

6. Auth Frontend Completion Summary

Category	Count / Status
Backend Auth APIs/routes available	20 total including aliases and health check
Important frontend-connected Auth APIs	17 connected directly or indirectly
Remaining important frontend features	0 major Auth features pending
Optional frontend-only improvements	Health status widget, dedicated address refresh button, optional GET aliases

7. Interview Readiness Notes

Ready from Auth POV: register/login for buyer and seller, Google OAuth, JWT access token, refresh token, 5 active sessions, logout current device, logout all devices, profile update, email verification, forgot/reset password, address management, rate limiting, security middleware, and user.created/user.updated events.

Frontend Ready: Auth UI contains complete user-facing flows for the important Auth APIs. The remaining items are optional utility screens, not blockers for placement/interview demo.

Demo flow suggestion: Register a buyer, verify email, login, update profile, add address, refresh session, logout all devices, then test forgot/reset password. For Google OAuth, open /login and click Google after backend env is configured.